

OTONOM MOBİL ROBOT SİMÜLASYONU

KAPSAMLI YAZILIM MİMARİSİ VE KULLANIM KILAVUZU

Geliştirici

Tuğran KILIÇ (Öğrenci No: 22406601056)

Bu belge, LiDAR tabanlı otonom navigasyon ve Sensör Füzyonu (EKF) projesinin altyapısını, matematiksel modellerini, Nesne Yönelimli Programlama (OOP) hiyerarşisini ve parametrik optimizasyon süreçlerini uçtan uca açıklayan resmi yazılım dokümantasyonudur.

İÇİNDEKİLER

1. Giriş ve Yazılımın Amacı
2. Sistem Gereksinimleri ve Geliştirme Ortamı
3. Yazılım Mimarisinin Temelleri ve OOP Hiyerarşisi
4. Bölüm 1: Ortam Modelleme Sınıfı (`Harita`) 4.1. Harita Sınırları ve Engel Tanımlamaları 4.2. Matplotlib Alt Eksen (Subplot) Yönetimi 4.3. Geometrik Çizimler ve Patch Nesneleri
5. Bölüm 2: Kinematik ve Çevre Algılama Modülü (`dif_suruslu_robot`) 5.1. Diferansiyel Sürüş Kinematiği ve Euler Ayrıklaştırması 5.2. Rotasyon Matrisleri ve Robotun Görselleştirilmesi 5.3. LiDAR Ray-Casting (Işın İzleme) Algoritması 5.4. Yapay Potansiyel Alanları (APF) ile Kontrol Hızının Belirlenmesi
6. Bölüm 3: Global Yörünge Planlama Modülü (`RRT` ve `dugum`) 5.1. Düğüm Veri Yapısı (Node Structure) 5.2. Rastgele Nokta Üretimi ve Hedef Eğilimi (Goal Bias) 5.3. Shapely Kütüphanesi ile Çarpışma Testleri

Gürültü Kovaryans Matrislerinin İnşası 7.2. Doğrusallaştırma (Jakobiyen) ve Tahmin (Prediction) Adımı 7.3. Kalman Kazancı ve Güncelleme (Update) Adımı

8. Bölüm 5: Ana Simülasyon Döngüsü (Main Loop) Analizi 8.1. Zaman İşleyişi ve Döngü Kısıtları 8.2. Sensör Gürültülerinin (Gaussian Noise) Simüle Edilmesi 8.3. Hata Metriklerinin (Öklid ve RMSE) Matematiksel Hesaplanması
9. Kapsamlı Parametre Optimizasyonu (Tuning Guide) 9.1. Harita ve Görev Konfigürasyonları 9.2. Robot Dinamikleri ve Sınır Değerleri 9.3. Algoritma Çekirdek Parametreleri

1. GİRİŞ VE YAZILIMIN AMACI

Bu dokümantasyon, `odev.py` isimli Python kaynak kodunun mimarisini, çalışma prensiplerini ve parametrik esnekliğini detaylandırmak üzere hazırlanmıştır. Geliştirilen yazılım, diferansiyel sürüş mekaniğine sahip otonom bir mobil robotun, statik engellerle dolu yapılandırılmamış bir 2B haritada hedefine güvenle ulaşmasını sağlayan tam kapsamlı bir simülasyon motorudur.

Yazılımın temel amacı, salt odometri kullanımından kaynaklanan "Sürüklenme (Drift)" hatasını matematiksel olarak kanıtlamak ve bu hatanın Genişletilmiş Kalman Filtresi (EKF) aracılığıyla LiDAR verileriyle kaynaştırılarak nasıl ortadan kaldırıldığını görselleştirmektir. Navigasyon problemi ise Hızlı Keşfeden Rastgele Ağaç (RRT) algoritmasının sağladığı global yönelim ile Yapay Potansiyel Alanları (APF) algoritmasının sağladığı dinamik yerel kaçınma reflekslerinin hibrit (kaskad) bağlanmasıyla çözülmüştür.

2. SİSTEM GEREKSİNİMLERİ VE GELİŞTİRME ORTAMI

Yazılımın derlenmesi ve hatasız çalışması için sistemde bulunması gereken kütüphane ve modüllerin teknik özellikleri aşağıda listelenmiştir.

- **Python Yorumlayıcısı:** Python 3.8 veya daha üstü bir sürüm gereklidir.
- **NumPy (`numpy`):** Vektörel işlemler, trigonometrik hesaplamalar, matris transpozisyonu, ters alma işlemleri (inversion) ve Gauss dağılımlı rastgele sayı üretimi (`np.random.normal`) için kullanılır.
- **Matplotlib (`matplotlib.pyplot` , `matplotlib.patches`):** Gerçek zamanlı grafik renderlama, `ion()` modunda interaktif çizim ve şekillerin (Polygon, Arrow) görselleştirilmesi için kullanılır.
- **Shapely (`shapely.geometry`):** RRT ve LiDAR algoritmalarının temelini oluşturan topolojik testler (kesişim - `intersects` , nokta - `Point` , çizgi - `LineString` , çokgen - `Polygon`) için kullanılan çekirdek geometri motorudur.

3. YAZILIM MİMARİSİNİN TEMELLERİ VE OOP HİYERARŞİSİ

Kod mimarisi tamamen Nesne Yönelimli Programlama (OOP) prensiplerine sadık kalınarak inşa edilmiştir. Spagetti kod (karmaşık yapı) oluşumunu engellemek amacıyla her bir matematiksel veya fiziksel olgu, kendi Sınıfı (Class) içerisine kapsüllenmiştir (Encapsulation). Proje dört ana sınıftan oluşur:

1. **Harita** : Fiziksel çevrenin ve Matplotlib UI (Kullanıcı Arayüzü) öğelerinin yöneticisi.
2. **dif_suruslu_robot** : Kinematik model, LiDAR sensörü ve APF kontrolörünün yöneticisi.
3. **RRT** : Global yol planlama algoritmasının yöneticisi.
4. **KalmanFiltresiEKF** : Sensör füzyonu, matris hesaplamaları ve lokalizasyonun yöneticisi.

4. BÖLÜM 1: ORTAM MODELLEME SINIFI (`Harita`)

gireceği sınırları belirler.

4.1. Harita Sınırları ve Engel Tanımlamaları

Kodun başlangıcında (Global Scope) harita sınırları ve engeller şu şekilde deklare edilmiştir:

```
x_lim = (0, 13)
y_lim = (0, 13)
engeller = [ [(1,7),(2,7),(2,9),(1,9)], ... ]
```

Harita sınıfının `__init__` (yapıcı) metodu, bu ham koordinat listelerini alır ve Shapely kütüphanesinin anlayabileceği matematiksel obje formatına (`Polygon`) çevirir. Bu dönüşüm işlemi, simülasyon sırasında RRT ve LiDAR'ın milyonlarca kesişim testini milisaniyeler içinde yapabilmesi için elzemdir.

4.2. Matplotlib Alt Eksen (Subplot) Yönetimi

Sınıf içerisindeki `cevre(self)` fonksiyonu, kullanıcının gördüğü 4 panelli dashboard'u (gösterge panelini) inşa eder.

- `plt.subplots(2, 2, figsize=(16, 12))` komutu ile 16'ya 12 inç boyutlarında bir pencere oluşturulur.
- Bu pencere `eks1` , `eks2` , `eks3` ve `eks4` olmak üzere 4 bağımsız çizim alanına bölünür.
- Başlangıç (`start_point`) ve Hedef (`finish_point`) noktaları `scatter` komutuyla sırasıyla yeşil ve kırmızı noktalar olarak haritaya sabitlenir.

4.3. Geometrik Çizimler ve Patch Nesneleri

`engel_ekle(self)` fonksiyonu, Shapely poligonlarını Matplotlib'in görselleştirme katmanına (`patches.Polygon`) dönüştürür. `facecolor="gray"` ve `edgecolor="black"` argümanlarıyla engeller gri dolgulu ve siyah çerçeveli olarak `eks1` üzerine çizdirilir.

5. BÖLÜM 2: KİNEMATİK VE ÇEVRE ALGILAMA MODÜLÜ

(`dif_suruslu_robot`)

Bu sınıf, simülasyonun ana aktörü olan robotun tüm fiziksel ve algısal hareketlerini kontrol eder. Diferansiyel bir sürüş modeline (iki bağımsız tekerlek) göre kısıtlanmıştır.

5.1. Diferansiyel Sürüş Kinematiği ve Euler Ayrıklaştırması

`hareket_ettir(self, x_local_hiz, omega)` fonksiyonu, robotun sürekli zamanlı fiziksel hareketlerini ayrık zamanlı ($\Delta t = 0.1$) simülasyona aktarır. Matematiksel olarak Euler sayısal entegrasyon formülü koda dökülmüştür:

```
self.global_orijin_x += (self.x_local_hiz) * (np.cos(self.bas_acisi)) * self.dt
self.global_orijin_y += (self.x_local_hiz) * (np.sin(self.bas_acisi)) * self.dt
self.bas_acisi += self.omega * self.dt
```

manevralarında açının $-\pi$ ile $+\pi$ sınırlarını aşarak trigonometrik hatalara (tekilliklere) neden olmasını engellemek için mükemmel bir normalizasyon işlemidir.

5.2. Rotasyon Matrisleri ve Robotun Görselleştirilmesi

`robotu_yerlestir(self, eks1)` fonksiyonu, robotun X ve Y koordinatlarına göre sadece bir nokta olarak değil, uzunluğu (0.4m) ve genişliği (0.25m) olan fiziksel bir dikdörtgen olarak çizilmesini sağlar. Robot döndükçe bu dikdörtgenin köşelerinin de dönmesi gerekir. Bu işlem, 3×3 'lük homojen rotasyon matrisi ile çözülmüştür:

```
self.rotasyon = np.array([
    np.cos(self.bas_acisi), -np.sin(self.bas_acisi), self.global_orijin_x],
    [np.sin(self.bas_acisi), np.cos(self.bas_acisi), self.global_orijin_y],
    [0, 0, 1])
```

Robotun yönünü belirten ok ise `eks1.arrow` fonksiyonuyla vektörel olarak konumlandırılır.

5.3. LiDAR Ray-Casting (Işın İzleme) Algoritması

Yazılımın en kompleks çevresel algılama fonksiyonu `lidar(self, poly_engeller, kapsam, isin_sayisi)` metodudur.

1. `isin_sayisi` değişkenine göre (örneğin 36), robotun merkezinden 360° etrafına, belirli bir menzile (`kapsam`) sahip ışınlar (`LineString`) oluşturulur.
2. Kod, bu ışınları haritadaki her bir engelle (`poly_engeller`) sırayla test eder (`isin.intersects(engel)`).
3. Eğer ışın engelin içinden geçiyorsa, engelin dış yüzeyi (boundary) ile ışının ilk kesişim noktası hesaplanır.
4. `robotun_merkezi.distance(nokta)` komutuyla, o açıda tespit edilen engelin robota olan mesafesi kaydedilir. Bu veriler `lidar_noktaları` listesi olarak dışarıya döner.

5.4. Yapay Potansiyel Alanları (APF) ile Kontrol Hızının Belirlenmesi

`apf_kontrol(self, ...)` fonksiyonu, robotun nereye gideceğini (çekim) ve neden kaçacağını (itme) fiziksel vektörlere dönüştürür.

- **Çekici Kuvvet Hesaplaması:** Hedef (X, Y) koordinatlarından robotun anlık (X, Y) koordinatları çıkarılarak oteleme vektörü bulunur. Bu vektör `cekme` (`k_att`) katsayısı ile çarpılır.
- **İtici Kuvvet Hesaplaması:** LiDAR'dan gelen engeller kontrol edilir. Eğer bir engel `etki_alani` 'ndan daha yakınsa, engelin tersi yönünde bir itici kuvvet (`1/mesafe - 1/etki_alani`) oluşturulur.
- **P-Kontrolör (Proportional):** Toplam X ve Y kuvvetleri birleştirilir, `np.arctan2` ile hedef açı (`aci_hata`) bulunur. Bu açı, 1.5 gibi bir oransal kazanç ile (P-Kontrol) açısal hıza (ω) dönüştürülür. Doğrusal hız (v) ise bu hata açısının kosinüsü alınarak yumuşatılır (Araç dönüş yaparken yavaşlar, düz yolda hızlanır).
- Son olarak `np.clip` fonksiyonu ile hesaplanan hızların, robotun fiziksel kapasitesini (`max_hiz` , `max_acisal_hiz`) aşması engellenir.

6. BÖLÜM 3: GLOBAL YÖRÜNGE PLANLAMA MODÜLÜ (`RRT` ve `dugum`)

RRT algoritması, uzayda "Ağaç (Tree)" veri yapısı üzerine kuruludur. Bunu sağlamak için `dugum` (Node) isimli yardımcı bir sınıf oluşturulmuştur. Bu sınıf her noktanın X ve Y koordinatını tutarken, `uye` (parent) değişkeni ile bir önceki bağlı olduğu düğümün referansını da hafızasında saklar. Bu sayede hedefe varıldığında geriye doğru (back-tracking) bir zincir oluşturulabilir.

6.2. Rastgele Nokta Üretimi ve Hedef Eğilimi (Goal Bias)

`rastgele_nokta(self)` fonksiyonu RRT'nin rastlantısallığını yönetir. Saf rastgelelik algoritmayı çok yavaşlatacağı için, "Goal Bias" (Hedef Eğilimi) isimli bir istatistiksel hile kullanılmıştır:

```
if random.random() > 0.2:
    return dugum(random.uniform(...))
return dugum(self.bitis_noktasi.x, self.bitis_noktasi.y)
```

Bu kod satırı, üretilen her 100 noktanın 20'sinin doğrudan "Hedef Nokta" olmasını sağlar. Ağaç böylece tamamen haritaya saçılmak yerine hızla hedefe doğru büyümeye teşvik edilir.

6.3. Shapely Kütüphanesi ile Çarpışma Testleri

Ağaç yeni bir noktaya uzanırken `carpisma_kontrol` fonksiyonu devreye girer. Eski düğüm ile yeni üretilen düğüm arasında bir çizgi (`LineString`) çekilir. `intersects` veya `contains` metotları kullanılarak bu çizginin herhangi bir poligon engele değip değmediği tespit edilir. Eğer çarpıyorsa, nokta ağaca dahil edilir (`self.dallar.append`). Algoritma hedef düğüme `adim_miktari` kadar yaklaştığında döngüyü sonlandırır ve `rota` listesini `::-1` ile ters çevirerek başlangıçtan hedefe doğru düzenli bir dizi koordinat (`waypoints`) döndürür.

7. BÖLÜM 4: SENSÖR FÜZYONU VE LOKALİZASYON MODÜLÜ

(`KalmanFiltresiEKF`)

Robotun konumu sadece odometri ile hesaplandığında zamanla sürüklenme (drift) yaşanır.

`KalmanFiltresiEKF` sınıfı, bu kaymayı LiDAR'dan gelen referans verilerle birleştirerek filtreler.

7.1. Durum Vektörü ve Gürültü Kovaryans Matrislerinin İnsası

Sınıfın `__init__` bloğunda filtre başlangıç matrisleri tanımlanır:

- `self.X` : Robotun $[x, y, \theta]^T$ durum (state) vektörü.
- `self.P` : Durum tahmininin hata kovaryans matrisi. 3×3 Birim matris (`np.eye(3)`) olarak başlar.
- `self.Q` : Sistem Gürültü Kovaryans Matrisi. Odometri hatalarını temsil eder. X ve Y için $(0.05)^2$, θ için $(2^\circ)^2$ varyans değerleri tanımlanmıştır.
- `self.R` : Ölçüm Gürültü Kovaryans Matrisi. LiDAR veya ortam eşleştirme (Scan Matching) sensörünün güvenilirliğini temsil eder, varyansı $(0.15)^2$ olarak set edilmiştir.

7.2. Doğrusallaştırma (Jakobiyen) ve Tahmin (Prediction) Adımı

`tahmin_et(self, v, w)` fonksiyonunda robotun Euler kinematik denklemleri işletilir. Ancak Kalman Filtresinin çalışabilmesi için sistemin F (Jakobiyen) matrisi ile doğrusallaştırılması gerekir:

```

r = np.array([
    1.0, 0.0, -v * np.sin(theta) * self.dt],
    [0.0, 1.0, v * np.cos(theta) * self.dt],
    [0.0, 0.0, 1.0]
])

```

Tahmin edilen pozisyonun belirsizliği (P matrisi) şu denklemle güncellenir: $\text{self.P} = \text{F} @ \text{self.P} @ \text{F.T} + \text{self.Q}$ (Burada $@$ sembolü NumPy'da matris çarpımını ifade eder).

7.3. Kalman Kazancı ve Güncelleme (Update) Adımı

`guncelle(self, z)` fonksiyonunda ise LiDAR'dan gelen ölçümler (z) sisteme dahil edilir.

- H matrisi, ölçüm modelidir. Sensör sadece X ve Y konumunu ölçebildiği için 2×3 'lük bir matristir.
- Sistem tahmini ile gerçek ölçüm arasındaki fark (Innovation/Yenilik) hesaplanır: $y = z - z_{\text{tahmin}}$.
- Kalman Kazancı $K = P @ H.T @ \text{np.linalg.inv}(S)$ denklemiyle hesaplanır (`np.linalg.inv` matrisin tersini alır).
- Robotun nihai konumu $X = X + (K @ y)$ ile güncellenerek mükemmel lokalizasyona (Sensör Füzyonu) ulaşılır.

8. BÖLÜM 5: ANA SİMÜLASYON DÖNGÜSÜ (MAIN LOOP) ANALİZİ

Dosyanın en altındaki `if __name__=="__main__":` bloğu, tanımlanan tüm bu sınıfları bir araya getirerek gerçek zamanlı animasyon döngüsünü başlatır.

8.1. Zaman İşleyişi ve Döngü Kısıtları

Öncelikle `Harita`, `dif_suruslu_robot`, `RRT` ve `KalmanFiltresiEKF` objeleri belleğe (RAM) yaratılır (Instance creation). `RRT` bir kereye mahsus çalıştırılıp rota elde edilir. Ardından `while True:` ile sonsuz bir döngü başlatılır. Bu döngünün kırılma (`break`) şartı, robotun merkez koordinatlarının hedef noktaya (`degisme_ ihtimali_olmayan_bitis_point`) 0.2 metreden daha fazla yaklaşmasıdır.

8.2. Sensör Gürültülerinin (Gaussian Noise) Simüle Edilmesi

Teorik formülleri gerçek dünyaya benzetmek için, robotun ideal APF kontrolünden çıkan v ve ω hızlarına rastgele Gauss gürültüleri (Gaussian Noise) eklenir:

```

v_gurultulu = v + np.random.normal(0, 0.02)
omega_gurultulu = omega + np.random.normal(0, 0.05)

```

Odometri (`dr_x`, `dr_y`, `dr_theta`) hesaplamaları, bu gürültülü (hatalı) tekerlek hızlarına göre yapılır, bu yüzden grafikteki turuncu çizgi zamanla gerçek yoldan kopar.

Benzer şekilde, LiDAR simülasyonundan gelen koordinatlara da gürültü eklenerek EKF'ye beslenir:

```

z_lidar_x = robot1.global_orijin_x + np.random.normal(0, 0.08)

```

8.3. Hata Metriklerinin (Öklid ve RMSE) Matematiksel Hesaplanması

Her döngüde robotun Gerçek Konumu (Ground Truth) ile Odometri tahmini ve EKF tahmini arasındaki hata Öklid uzaklığı (`np.hypot(gt_x - tahmin_x, gt_y - tahmin_y)`) denkleminde hesaplanarak `err_dr_list` ve `err_ekf_list` dizilerine eklenir.

Kümülatif performansı ölçmek için ise RMSE (Root Mean Square Error - Kök Ortalama Kare Hata) sürekli olarak güncellenir:

```
rmse_ekf_list.append(np.sqrt(np.mean(np.square(err_ekf_list))))
```

Bu diziler, kodun geri kalanında `harita1.eks2`, `harita1.eks3` ve `harita1.eks4` komutlarıyla ekrandaki 4 panelli dashboard üzerinde anlık olarak grafiklere dökülür. Döngünün sonunda yer alan `plt.pause(0.01)` komutu grafikleri render eder ve döngüyü 0.01 saniye bekleterek işlemci kitlenmesini (freeze) önler.

9. KAPSAMLI PARAMETRE OPTİMİZASYONU (TUNING GUIDE)

Bu dokümantasyonun amacı, yazılımı geliştirmek veya farklı senaryolarda test etmek isteyen mühendisler için bir referans noktası olmaktır. Aşağıdaki tablolar ve listeler, `odev.py` içerisinde değiştirilebilir tüm katsayıları, konumlarını ve fiziksel etkilerini tanımlar.

9.1. Harita ve Görev Konfigürasyonları

- `baslangic_point = (1, 1)` (Satır ~8): Robotun doğma noktasıdır. Değiştirildiğinde haritadaki engellerin içine denk gelmemesine dikkat edilmelidir.
- `bitis_point = (10, 10)` (Satır ~9): Robotun gitmeye çalışacağı koordinat.
- `x_lim`, `y_lim` (Satır ~19): Haritanın genişliğini ve yüksekliğini belirler. Standart olarak 13x13 metredir. Harita büyütülürse Matplotlib eksenleri otomatik ölçeklenir (Auto-scaling).
- `engeller` Dizisi (Satır ~22): İç içe listelerden oluşur. Her bir liste `[(X1,Y1), (X2,Y2), (X3,Y3)...]` formunda bir engeli temsil eder. Buraya dilediğiniz kadar asimetrik şekil (Çokgen) ekleyebilirsiniz.

9.2. Robot Dinamikleri ve Sınır Değerleri

- `genislik = 0.25`, `uzunluk = 0.4` (Satır ~13): Robotun geometrik boyutlarıdır. Büyütülmesi halinde APF `etki_alani` parametresinin de büyütülmesi gerekir, aksi takdirde robotun köşeleri engellere sürtünür.
- `dt = 0.1` (Satır ~17): Diferansiyel integrasyon süresidir. 0.1 saniye, sistemin saniyede 10 kere (10 Hz) hesaplama yaptığı anlamına gelir. Bu değeri 0.05 (20 Hz) yapmak simülasyonu daha yavaş ama çok daha pürüzsüz ve hatasız hale getirir.
- `max_hiz = 0.25`: Ana döngüdeki `robot1.apf_kontrol(...)` argümanı olarak gönderilir. Robotun m/s cinsinden en yüksek düz gidiş hızıdır. Yükseltilmesi sistemin ataletini artırarak RRT köşelerini dıştan almasına neden olur.
- `max_acisal_hiz = np.pi/1.3`: Robotun olduğu yerde dönme hızıdır (Radyan).

9.3. Algoritma Çekirdek Parametreleri

çok yoğun ve karmaşık olur, hesaplama uzar. 2.0 yapılırsa ağaç engellerin arasından geçemeyebilir.

- `max_iterasyon=4000` (**RRT Sınıfı**): RRT'nin çözüm bulana kadar en fazla kaç nokta üreteceğidir. Çok büyük haritalarda bu değeri 10000'lere çekmek gerekebilir.
- `cekme=5, itme=0.3` (**APF Fonksiyonu**): Robotun hedefe çekilme ve engelden kaçma arzularını belirtir. Robot çok fazla duvara çarpıyorsa `itme` parametresini 1.0 veya 1.5 yapabilirsiniz.
- `etki_alani=0.09` (**APF Fonksiyonu**): Robot engeli LiDAR ile görse bile, aradaki mesafe 0.09